Evolução da Qualidade Estrutural de Projetos Spring: transição para Spring Boot e evidências de assistência por IA

Gabriel Ferreira[1], **Guilherme Augusto**[1]

[1]Engenharia de Software
Pontifícia Universidade Católica de Minas Gerais (PUC-MG)
Belo Horizonte – MG – Brazil

{gabriel.amaral.1447804,gajsouza}@sga.pucminas.br

Abstract. *This observational longitudinal study characterizes the structural evolution of public Java projects in the Spring ecosystem, focusing on repositories created between 2009 and 2013. Annual snapshots from 2009 to 2025 and a separate partial observation in 2026 are recovered from pinned Git histories. Seven measures are extracted with the CK tool and an audited adapter, retaining accessible inactive and archived repositories. The frozen sample contains 1,616 candidates; 388 projects contribute 2,533 eligible annual observations. The oldest cohort comprises 52 eligible projects and 609 observations. Arithmetic means, individual trajectories and within-project changes describe coupling, complexity, size, inheritance and cohesion. Sixteen oldest-cohort projects have complete-year before/after pairs around the first observable presence of Spring Boot. AI evidence supports only one complete-year pair and ten partial-year pairs in that cohort, making this analysis exploratory. The observed dimensions vary in different directions. These descriptions do not establish uniform improvement, causal effects of Boot or AI, or the maturity of the broader developer community. Absence of detectable AI traces is not interpreted as non-use.*

Resumo. *Este estudo observacional longitudinal caracteriza a evolução estrutural de projetos Java públicos do ecossistema Spring, com foco nos repositórios criados entre 2009 e 2013. Recuperam-se versões anuais de 2009 a 2025 e uma observação parcial de 2026, apresentada separadamente, a partir de históricos Git com referências fixadas. Sete medidas são extraídas pela ferramenta CK e por um adaptador auditado, mantendo projetos inativos e arquivados acessíveis. A amostra congelada contém 1.616 candidatos; 388 projetos contribuem com 2.533 observações anuais elegíveis. A coorte antiga reúne 52 projetos elegíveis e 609 observações. Médias aritméticas, trajetórias individuais e mudanças dentro dos projetos descrevem acoplamento, complexidade, tamanho, herança e coesão. Dezesseis projetos antigos possuem pares antes/depois em anos completos em torno da primeira presença observável de Spring Boot. As evidências de IA permitem apenas um par completo e dez pares parciais nessa coorte, tornando essa análise exploratória. As dimensões observadas variam em direções distintas. Essas descrições não estabelecem melhoria uniforme, efeitos causais de Boot ou IA, nem a maturidade da comunidade de desenvolvedores. Ausência de vestígios detectáveis de IA não é interpretada como ausência de uso.*

Bacharelado em Engenharia de *Software* - PUC Minas
Trabalho de Conclusão de Curso (TCC)

Orientador de conteúdo (TCC I): Danilo Maia - danilomaia@pucminas.br
Orientador de conteúdo (TCC I): João Pedro Batisteli - joaobatisteli@pucminas.br
Orientador de conteúdo (TCC I): Leonardo Vilela - leonardocardoso@pucminas.br
Orientador acadêmico (TCC I): Cleiton Tavares - cleitontavares@pucminas.br
Belo Horizonte, versão de trabalho de 11 de outubro de 2026.

1. Introdução

A qualidade interna de *software* descreve propriedades estruturais do código-fonte, como modularidade, coesão e acoplamento, que podem ser investigadas por análise estática [ISO/IEC 2011, Pressman and Maxim 2020]. Em sistemas orientados a objetos, a suíte de Chidamber e Kemerer oferece indicadores para caracterizar essas dimensões [Chidamber and Kemerer 1994]. Esses indicadores não constituem, isoladamente, uma nota de maturidade: sua interpretação depende do tamanho, do domínio e da organização do projeto.

O ecossistema Spring permite investigar a evolução de projetos Java ao longo de diferentes períodos tecnológicos. Projetos iniciados antes da consolidação do Spring Boot podem ter atravessado mudanças de configuração, dependências, práticas de desenvolvimento e ferramentas de assistência. O Spring Boot 1.0 estável foi lançado em 1ž de abril de 2014 [Spring 2014]; por isso, o período de 2009 a 2013 é tratado como contexto de Spring tradicional, com identificação explícita de eventuais versões preliminares de Boot. A presença atual de Boot em um repositório criado em 2010 não demonstra que essa tecnologia já integrava seu código em 2010.

A mesma distinção é necessária para a inteligência artificial. Um projeto antigo pode apresentar hoje vestígios de ferramentas de IA introduzidos recentemente. Datas de criação dos repositórios não datam o uso dessas ferramentas, e fronteiras de calendário não separam perfeitamente produção humana e assistida. O Copilot já estava em prévia técnica em junho de 2021 [GitHub 2021]. Evidências de arquivos, identidades e mensagens de commit permitem observar parte dessa adoção, mas não todo uso nem a proporção do código produzida por IA [Robbes et al. 2026].

A pergunta central deste trabalho é: **como a qualidade estrutural de projetos do ecossistema Spring evoluiu ao longo do tempo, e quais mudanças acompanharam a adoção do Spring Boot e o aparecimento de evidências de assistência por IA?** O foco está no acompanhamento dos mesmos projetos, especialmente aqueles criados entre 2009 e 2013. As demais coortes oferecem contexto sobre projetos surgidos em épocas posteriores. Esse desenho distingue evolução dentro do projeto de diferenças de composição entre conjuntos de projetos.

O objetivo geral é caracterizar longitudinalmente a evolução estrutural dos projetos selecionados por versões anuais recuperadas do histórico Git. Os objetivos específicos são: (i) descrever trajetórias de acoplamento, complexidade, tamanho, herança e coesão; (ii) descrever mudanças nos mesmos projetos em torno da primeira presença observável de Spring Boot; (iii) explorar as mudanças que acompanham o aparecimento de evidências de IA; e (iv) situar essas trajetórias no contexto dos projetos disponíveis em

cada época, por tamanho, framework e coorte de criação. O paradigma GQM liga essas perguntas às sete medidas extraídas pela ferramenta CK. O acompanhamento histórico constitui o eixo principal; Boot e evidências de IA são marcos observáveis cuja análise depende da cobertura disponível.

O interesse na evolução da comunidade é abordado por esse recorte de projetos, sem tratar a amostra como representação de todos os usuários de Spring. A idade de um repositório, a adoção de uma ferramenta e a redução de uma métrica não são equivalentes a maturidade. Neste trabalho, qualidade estrutural designa as dimensões do código Java observadas pelos indicadores; não inclui uma avaliação direta de produtividade, funcionalidade, desempenho ou competência dos desenvolvedores.

A relevância do estudo está em produzir evidências históricas rastreáveis, com código, datas, versões e cobertura associados a cada medição. A mineração de repositórios possibilita esse acompanhamento, mas exige cuidado com seleção, disponibilidade do histórico e interpretação dos metadados [Hassan 2008, Kalliamvakou et al. 2016]. Mudanças de framework, linguagem, domínio, equipe e tamanho são explicações concorrentes para variações das métricas. Nem a injeção de dependências elimina a influência do framework sobre a estrutura, nem um contraste entre períodos permite atribuir diferenças exclusivamente ao desenvolvedor ou à IA.

Os resultados incluem tabelas de cobertura, trajetórias anuais, médias por época e diferenças entre versões do mesmo projeto, com registros de reprodução. A investigação admite estabilidade, aumentos e reduções das métricas, sem presumir melhoria. A Seção 2 apresenta a fundamentação; a Seção 3, os trabalhos relacionados; a Seção 4, o protocolo histórico; a Seção 5, os resultados e sua discussão; e a Seção 6, as respostas às perguntas do estudo e os desdobramentos ainda necessários.

2. Fundamentação Teórica

Esta seção apresenta os conceitos que embasam o estudo: qualidade estrutural de *software*, métricas orientadas a objetos, o paradigma *Goal-Question-Metric*, o acompanhamento histórico por mineração de repositórios e as evidências de assistência por inteligência artificial.

2.1. Qualidade interna e métricas orientadas a objetos

A norma *ISO/IEC 25010* define qualidade de *software* como um conjunto de características que expressam o grau em que um sistema atende a necessidades explícitas e implícitas [ISO/IEC 2011]. Nesse modelo, a qualidade interna refere-se a propriedades do código-fonte observáveis por análise estática, incluindo modularidade, coesão e facilidade de manutenção. Pressman e Maxim destacam que sistemas com melhor qualidade interna tendem a exigir menor esforço de evolução e a permanecer viáveis por mais tempo [Pressman and Maxim 2020].

Para tornar essas características mensuráveis em código orientado a objetos, Chidamber e Kemerer propuseram um conjunto clássico de indicadores em nível de classe [Chidamber and Kemerer 1994]. Acoplamento entre objetos (*Coupling Between Objects* — CBO) descreve o grau de dependência de uma classe em relação a outras. Métodos ponderados por classe (*Weighted Methods per Class* — WMC) agrega a complexidade dos métodos. Profundidade da árvore de herança (*Depth of Inheritance Tree* — DIT)

e número de filhos (*Number of Children* — NOC) caracterizam a hierarquia. Resposta de uma classe (*Response for a Class* — RFC) estima o conjunto de métodos potencialmente acionados a partir de uma mensagem. Falta de coesão em métodos (*Lack of Cohesion in Methods* — LCOM) indica o quanto os métodos de uma classe operam sobre conjuntos disjuntos de atributos. Essas métricas passaram a ser amplamente empregadas para caracterizar projeto, manutenção e falhas em sistemas orientados a objetos [Singh et al. 2010, Radjenović et al. 2013, Aniche et al. 2016]. A forma operacional em que são extraídas neste trabalho, incluindo a ferramenta e as variantes adotadas, é especificada na Seção 4.

Neste estudo, WMC representa a complexidade estrutural agregada dos métodos, conforme a implementação do CK; não mede tempo de execução, consumo de recursos ou desempenho computacional. O CBO empregado é o indicador geral de dependências entre classes do CK, sem uma separação implementada entre acoplamento horizontal e vertical. Para coesão, utiliza-se a coluna `lcom*`, variante normalizada documentada pelo instrumento, e não a coluna `lcom` original. As seis medidas em nível de classe são complementadas por LOC por método; essa medida de tamanho não é apresentada como uma sétima integrante da suíte CK original.

Valores devem ser interpretados em conjunto e segundo a função do código. Uma redução de DIT ou NOC descreve uma mudança na hierarquia, sem estabelecer que o projeto se tornou melhor. Mesmo mudanças em complexidade ou acoplamento exigem contexto sobre responsabilidades e tamanho [Aniche et al. 2016]. Nenhuma métrica isolada prova violação de um princípio de projeto, nem identifica a autoria humana ou assistida do código.

2.2. Goal-Question-Metric

O paradigma *Goal-Question-Metric* (GQM) organiza avaliações de *software* em três níveis: o objetivo do estudo (*goal*), as perguntas que o detalham (*questions*) e as métricas que as respondem (*metrics*) [Basili et al. 1994]. A abordagem evita coletar indicadores sem vínculo com a intenção da pesquisa: primeiro define-se o que se deseja aprender; em seguida, traduz-se esse objetivo em perguntas mensuráveis; por fim, escolhem-se as medidas adequadas. Neste trabalho, o GQM liga o objetivo de analisar a evolução da qualidade estrutural em projetos do ecossistema Spring às dimensões de acoplamento, complexidade, herança, coesão e tamanho, operacionalizadas na Seção 4.

2.3. Mineração de Repositórios de Software

A Mineração de Repositórios de *Software* (*Mining Software Repositories* — MSR) analisa artefatos reais — código-fonte, histórico de versões e metadados de plataformas — para identificar padrões de desenvolvimento, evolução e qualidade [Hassan 2008]. Em vez de apoiar-se apenas em relatos ou documentação, a MSR observa evidências extraídas dos próprios repositórios. No GitHub, essa estratégia exige filtros de relevância, exclusão de *forks* e distinção entre criação na plataforma, idade do código e disponibilidade de versões, a fim de evitar a interpretação de metadados atuais como evidência histórica [Kalliamvakou et al. 2014, Kalliamvakou et al. 2016]. Esses cuidados conceituais orientam a amostragem descrita na metodologia.

2.4. Inteligência artificial generativa

Neste trabalho, inteligência artificial generativa designa sistemas baseados em modelos de linguagem de grande escala, treinados em corpora de texto e código, capazes de produzir ou completar trechos de programa a partir de contexto e de *prompts*. Ferramentas como GitHub Copilot, ChatGPT, Claude Code e Cursor ilustram formas distintas de assistência, introduzidas em momentos diferentes. E4, iniciada em 2022, é um recorte de calendário do estudo; pertencer a essa época não confirma uso de IA, assim como estar em E3 não o exclui.

Experimentos sobre produtividade e qualidade de sugestões motivam investigar essas ferramentas, mas medem resultados diferentes das métricas estruturais de um repositório [Peng et al. 2023, Moradi Dakhel et al. 2023]. Também é necessário distinguir assistentes de completamento e agentes de codificação: estes podem deixar rastros mais explícitos, como coautoria em commits [Robbes et al. 2026]. Consequentemente, a distribuição temporal dos vestígios encontrados depende tanto da adoção quanto da possibilidade de detectá-la. O estudo acompanha evidências observáveis, sem equipará-las a todo uso de IA.

3. Trabalhos Relacionados

Os trabalhos relacionados são organizados em três frentes: acompanhamento histórico de sistemas, interpretação de métricas estruturais e evidências de assistência por IA. Essa organização situa a contribuição principal do TCC na evolução de projetos Spring, tratando a IA como uma possível mudança posterior em suas trajetórias.

3.1. Evolução de sistemas e mineração de repositórios

[Godfrey and Tu 2000] acompanharam 96 versões do núcleo Linux, investigando o crescimento do sistema e de seus subsistemas. O estudo encontrou trajetórias que exigiam examinar a organização interna, além dos valores agregados de tamanho. A relação com este TCC é metodológica: observar versões do mesmo sistema permite investigar mudanças que uma única medição atual não revela. O objeto e as medidas diferem, pois o presente trabalho acompanha vários projetos Java/Spring por snapshots anuais e métricas estruturais, sem transferir o padrão de crescimento do Linux para esse ecossistema.

A mineração de repositórios oferece meios para reconstruir essas trajetórias, mas a disponibilidade de dados não garante sua adequação à pergunta de pesquisa [Hassan 2008]. Os estudos de [Kalliamvakou et al. 2014, Kalliamvakou et al. 2016] discutem problemas de interpretação dos repositórios do GitHub e de sua relação com a atividade de desenvolvimento. Esse cuidado fundamenta a distinção, neste trabalho, entre um candidato encontrado por metadados, um histórico recuperável e uma versão tecnicamente elegível. A retenção de inativos e arquivados evita uma exclusão explícita por atividade recente, embora não elimine as perdas de projetos que já desapareceram.

3.2. Interpretação contextual das métricas

A abordagem SATT, de [Aniche et al. 2016], propõe limites de métricas específicos para papéis arquiteturais, em vez de avaliar todas as classes pelos mesmos valores de referência. O estudo examinou projetos MVC e Android e relacionou diferenças de métricas às responsabilidades das classes. Para este TCC, a implicação é que uma variação agregada

pode refletir mudanças na composição arquitetural do projeto. O trabalho não aplica os limites SATT nem classifica automaticamente os projetos como bons ou ruins: utiliza as mesmas medidas ao longo do tempo e explicita o contexto de sua interpretação.

No contexto de injeção de dependências, [Sun et al. 2022] propõem medidas que distinguem formas de acoplamento relevantes à manutenibilidade. A investigação mostra por que métricas de dependência precisam ser interpretadas à luz dos mecanismos utilizados no código. Este TCC mantém o CBO geral do instrumento CK; portanto, não reproduz as medidas ponderadas daquele estudo nem assume que a injeção de dependências elimina o acoplamento observado. A comparação de variantes de CBO em [Child et al. 2019] também reforça a necessidade de documentar qual definição foi operacionalizada.

Esses trabalhos delimitam o alcance das conclusões: métricas estruturais descrevem propriedades do código, enquanto sua relação com qualidade depende do contexto. A consistência do extrator entre versões ajuda a comparar medições, mas não transforma uma redução numérica em evidência suficiente de melhoria da arquitetura.

3.3. Assistência por IA e identificação de vestígios

Estudos experimentais sobre Copilot e outros geradores analisam produtividade, correção e qualidade em tarefas delimitadas [Peng et al. 2023, Moradi Dakhel et al. 2023, Yetistiren et al. 2023]. O EvalPlus, apresentado por [Liu et al. 2023], amplia os testes usados para avaliar a correção funcional de soluções geradas. Esses resultados motivam a investigação da assistência por IA, mas não fornecem uma previsão direta para as métricas CK de sistemas completos. O presente estudo observa repositórios históricos; não reproduz os experimentos nem mede tempo de desenvolvimento ou correção funcional.

A abordagem BIMAN, de [Dey et al. 2020], combina características de nomes, mensagens e atividade para identificar bots que realizam commits. Essa identificação não equivale à detecção de assistência por IA: automações convencionais também produzem commits, e contribuições assistidas podem permanecer sob a identidade do desenvolvedor. Por isso, um autor automatizado precisa de interpretação contextual e a ausência de uma identidade de bot não caracteriza um projeto como exclusivamente humano.

[Robbes et al. 2026] investigam vestígios de agentes de codificação em 128.018 projetos do GitHub, distinguindo-os de assistentes de completamento. O estudo explora rastros como configuração e coautoria, que tornam parte dessa adoção observável. O detector deste TCC é uma adaptação local, com heurísticas próprias e inspeção temporal do histórico Git; sua precisão não é herdada daquele trabalho. Sinais de configuração, identidades e trailers são conservados como evidências distintas, sem determinar a parcela de código gerada ou a continuidade do uso.

A contribuição deste trabalho está na combinação entre versões históricas de projetos Spring, medições estruturais rastreáveis e marcos observáveis de framework e assistência por IA. A evolução dos projetos constitui o objeto principal. Os contrastes em torno de Boot e IA complementam essa descrição, sujeitos à quantidade e à qualidade das versões e evidências disponíveis.

4. Materiais e Métodos

O estudo é empírico, quantitativo, observacional e longitudinal, com síntese descritiva. A unidade de acompanhamento é o repositório; a unidade de medição é uma versão anual de seu código. A análise principal acompanha projetos criados entre 2009 e 2013, e as demais coortes oferecem contexto. O acompanhamento de evidências de IA é exploratório, condicionado à existência de versões elegíveis antes e depois do primeiro vestígio. As conclusões descrevem mudanças observadas. Uma sequência antes/depois não identifica, por si só, um efeito causal de Spring Boot ou de IA.

4.1. Ambiente e instrumentos

O instrumento `analisar_evolucao.py` separa recontagem, descoberta, planejamento, coleta e relatório. A descoberta utiliza a API REST de busca do GitHub, com paginação particionada e controle de limites. Os filtros e o sorteio operam sobre metadados salvos. A clonagem e a inspeção de evidências de IA são posteriores ao congelamento da amostra; a inspeção histórica usa objetos Git locais, sem requisições por snapshot.

A extração reutiliza CK 0.7.1-SNAPSHOT do commit `8a1ef916`, com identificador completo registrado no manifesto do instrumento, JDT 3.44.0 e o adaptador auditado que preserva a chave arquivo e classe. A JVM, os parâmetros, os hashes do JAR, do adaptador e dos scripts são registrados e congelados. Nenhum projeto amostrado é compilado ou executado, nem suas dependências são baixadas. As fórmulas permanecem as do CK; dependências ausentes e resolução parcial de tipos continuam sendo limitações [Aniche 2015, Child et al. 2019].

4.2. Descoberta, filtros e sorteio

Reaproveita-se a população original de 95.894 repositórios da consulta `topic:spring-boot fork:true`, salva antes dos filtros em setembro de 2026. Essa população é unida às consultas por tópicos `spring`, `spring-framework` e `spring-mvc`, com deduplicação pelo identificador do GitHub. A procedência e as datas de observação são preservadas, sem apresentar a união como um censo simultâneo. Tópicos indicam candidatos; a confirmação de Spring ocorre no conteúdo das versões históricas.

Aplicam-se localmente filtros de linguagem principal Java, forks, espelhos, repositórios desabilitados, templates, vazios e materiais claramente não relacionados a desenvolvimento de software. Os critérios operacionais de nomes e tópicos são preservados em configuração, e casos educacionais ambíguos permanecem sinalizados para revisão. Arquivamento e falta de atividade recente não excluem candidatos. Projetos que deixaram de estar acessíveis são registrados como falhas de recuperação, sem substituição silenciosa.

As coortes de criação são E1 (2009–2013), E2 (2014–2017), E3 (2018–2021) e E4 (2022–2026, até o corte da execução). A data de criação estratifica o sorteio; a data do snapshot determina a época de cada medição. Para cada coorte, ordenam-se candidatos pelo SHA-256 de `20261011:identificador`, selecionando até 500; quando houver menos, incluem-se todos. A semente e a lista são congeladas antes das métricas. Os primeiros dez sorteados de cada coorte formam o piloto, com até 40 candidatos. Não se substituem candidatos por falha, tamanho ou resultado CK. O teto por coorte produz

frações amostrais diferentes: as médias conjuntas descrevem a amostra observada, sem ponderação para estimar a distribuição de toda a população do GitHub.

Depois de fixar a amostra e o piloto, ordena-se o processamento por atividade recente, antiga e data desconhecida. Essa ordem é operacional e não altera pertencimento à amostra. Antes da clonagem, publicam-se população, exclusões e candidatos por coorte, com comprovação de retenção de inativos e arquivados.

4.3. Recuperação das versões históricas

Cada repositório é clonado uma vez com o histórico completo alcançável da branch padrão, sem limite de profundidade ou de 30 commits. A referência e seu SHA são fixados. O histórico é percorrido pelo primeiro pai dos merges, representando a linha principal de integração [Git 2026]. Para cada encerramento de ano em UTC, seleciona-se o último commit dessa linha com data de committer anterior ou igual ao corte. Registra-se também a data de autoria. A data Git do committer é uma aproximação operacional da integração, não prova de quando o conteúdo foi publicado no GitHub.

Medem-se os anos completos de 2009 a 2025. O ano de 2026 usa o instante fixado no início da execução e é apresentado separadamente como período parcial. Sem novos commits, reutiliza-se o SHA anterior e marca-se "sem novo commit". Sem histórico anterior ao corte, registra-se ausência de snapshot. Datas não monotônicas, autoria futura, ancestrais posteriores ao corte e histórico raso ou incompleto geram diagnósticos; nenhum caso é substituído pelo código atual. Históricos anteriores à criação do repositório no GitHub são sinalizados como possíveis importações ou migrações.

Cada observação conserva identificador, ano, corte, SHA, datas, framework, versões declaradas, cobertura, métricas e motivos de exclusão. Extrações idênticas são reutilizadas pelo hash do inventário de fontes Java e do instrumento. As evidências de IA mantêm uma chave temporal própria, pois árvores iguais podem ter históricos distintos. Resultados concluídos são reutilizados na retomada; erros permanecem explícitos.

4.4. Framework e evidências de assistência por IA

A identificação técnica inspeciona declarações Maven e Gradle disponíveis no snapshot, distinguindo dependências de produção, testes, gerenciamento, catálogos e plugins. Referências locais de propriedades e versões são resolvidas quando possível; valores não resolvidos são preservados. Um catálogo sem uso não confirma adoção. Dependências exclusivamente de teste não confirmam Boot no código de produção. Código do próprio framework Spring é identificado separadamente e não integra a comparação principal de projetos que utilizam Spring. Declarações de perfis são registradas como condicionais; a análise estática não confirma a configuração efetivamente executada.

Um complemento de classificação resolve usos efetivos de mapas Groovy em scripts Gradle locais referenciados por `apply from`, incluindo indireções para catálogos TOML. As declarações são lidas no mesmo SHA; expressões dinâmicas e referências ambíguas permanecem sem resolução. O complemento tem hash próprio, preservando a identificação do produtor CK e seus valores brutos. Essa emenda está identificada na execução derivada, sem ser apresentada como componente do instrumento inicialmente congelado.

A detecção de IA é uma adaptação local inspirada em [Robbes et al. 2026]. Após a seleção, inspecionam-se arquivos de configuração presentes na versão, entradas de gitignore, identidades Git de autor e trailers de coautoria nos commits alcançáveis. A inspeção de commits não se limita aos 30 mais recentes. Evidências de branches laterais só se tornam disponíveis quando integradas à linha principal. As mudanças dos caminhos de configuração ao longo do primeiro pai permitem preservar sinais observados em versões intermediárias, mesmo se removidos depois.

Dois estados são conservados: sinais presentes no snapshot e evidências já observadas no histórico disponível até ele. A inspeção de arquivos segue a lista de caminhos do detector, predominantemente a partir da raiz; arquivos equivalentes em caminhos não previstos podem não ser encontrados. O primeiro vestígio observado não é apresentado como a data exata do primeiro uso. "Sem evidência detectada" não equivale a "sem IA"; configurações copiadas, gitignore e nomes genéricos podem ser ambíguos. As primeiras evidências e transições dos projetos antigos do piloto passam por inspeção dos objetos Git, com identificação de quem realizou a revisão. A precisão do estudo original não é atribuída automaticamente ao detector adaptado.

Os tipos de vestígio têm interpretações diferentes. Uma coautoria atribuída a um agente vincula uma declaração de assistência a um commit; um arquivo de instruções registra configuração ou intenção de uso; uma entrada no gitignore pode apenas prever arquivos de uma ferramenta. Nenhum desses sinais, isoladamente, mede a quantidade de código assistido. A classificação agregada conserva esses vestígios sob a expressão "com evidência detectada", sem conferir a todos a mesma força como demonstração de uso.

4.5. Métricas e operacionalização GQM

O instrumento conserva sete medidas: médias por classe de CBO geral, RFC, WMC, DIT, NOC e LCOM*, além da média de LOC por método. Na implementação CK adotada, RFC conta chamadas distintas identificadas pelo visitante; WMC agrega complexidade ciclomática e não mede desempenho de execução. LCOM* corresponde à coluna `lcom*`, preservando-se `lcom` original apenas como auxiliar. Dados brutos por classe e método e agregações de produção e de testes são conservados.

O corte principal de 2.000 LOC é aplicado a cada snapshot, com linhas físicas Java contendo código, sem comentários ou linhas vazias e sem contar arquivos repetidamente por classes internas. A versão principal inclui fontes e testes, com sensibilidade somente de produção. Versões menores continuam registradas e não excluem toda a trajetória do projeto. Cobertura incompleta, métricas ausentes e erros impedem a participação da versão na comparação principal; ausência de dados não é convertida em zero. Submódulos não inspecionados e diagnósticos de configuração também permanecem explícitos.

O limiar de 2.000 LOC delimita operacionalmente o porte das versões incluídas; não é um valor validado de maturidade ou qualidade. Ele pode excluir fases iniciais de projetos e aplicações pequenas, ainda que legítimas. As conclusões ficam condicionadas a esse recorte. A comparação com outros limiares constitui uma análise adicional ainda não realizada nesta execução.

Tabela 1. Objetivos, questões e medidas do desenho histórico (GQM).

Q	Questão e objetivo	Medidas e evidências
Q1	Como a estrutura dos mesmos projetos evolui anualmente? Caracterizar trajetórias, sem presumir melhoria.	Sete medidas CK; LOC; cobertura; idade do projeto.
Q2	Quais mudanças acompanham a primeira presença observável de Boot? Descrever antes/depois e projetos sem transição observada.	Sete medidas; framework e versão declarada; SHA anterior e posterior.
Q3	Quais mudanças acompanham os vestígios de IA nos casos observáveis? Explorar os pares disponíveis, separando presença atual e evidência histórica.	Sete medidas; caminhos, trailers, datas e primeira evidência observada.
Q4	Como essas trajetórias se relacionam com projetos da mesma época? Distinguir evolução interna e composição dos grupos.	Médias por repositório e época; quantidades; estratos de LOC, framework e coorte.

4.6. Síntese descritiva e comparações

Apresentam-se quantidades de projetos e snapshots, médias, mudanças entre versões e gráficos de trajetória. Nas sínteses por época, calcula-se primeiro a média aritmética das observações anuais do repositório naquela época e, depois, a média aritmética entre repositórios. Assim, cada projeto tem uma contribuição, independentemente de quantos snapshots possui. Mudanças de evidência de IA dentro de uma época são apresentadas como transição observada, sem duplicar o projeto entre grupos. O período parcial de 2026 permanece separado.

As médias por época descrevem os projetos elegíveis em cada período, cujos conjuntos podem mudar. Elas não constituem um pareamento entre épocas nem isolam mudanças dos mesmos projetos. A atribuição de pesos iguais evita que o número de snapshots determine o peso do repositório, mas não controla diferenças de tamanho, domínio ou arquitetura. As médias também podem ser influenciadas por valores extremos; seu sinal não informa quantos projetos aumentaram ou diminuíram uma métrica.

A permanência de um SHA em anos sem novos commits descreve a estrutura disponível no projeto ao final do período, e não código produzido naquele ano. A síntese da amostra inclui esse legado. A marcação de versões repetidas permite distinguir estabilidade por inatividade de mudanças estruturais observadas em projetos que continuaram evoluindo.

Nas comparações antes/depois, utilizam-se a última versão anual elegível anterior ao marco e a primeira elegível posterior. Projetos já positivos na primeira observação são marcados como censurados à esquerda: sua transição não foi observada. A transição para Boot exige uma versão anterior com Spring confirmado. Apresentam-se os valores antes/depois e a média das diferenças individuais; percentuais não são calculados quando o denominador é zero. Lacunas entre anos são informadas. O contexto inclui projetos sem a mesma transição e estratos de tamanho, framework e coorte, sem afirmar equivalência de domínio ou causalidade.

O relatório histórico principal não apresenta p-valores ou deltas de Cliff. Testes de grupos independentes dos pilotos transversais não validam este painel de medições repetidas. Inferência futura, se proposta, demandará um desenho específico que considere dependência dentro do repositório, confundimento temporal e perdas de cobertura; ela não

integra a apresentação descritiva atual.

Um complemento identificado separadamente apresenta pares cuja primeira observação elegível posterior ao marco está disponível apenas no corte parcial de 2026. Esses pares não são reunidos aos de anos completos. O complemento conserva os SHA, a data do corte e um manifesto próprio de apresentação, sem alterar a extração CK.

4.7. Validação e ampliação

A validação verifica recontagem e invariância do sorteio à atividade, limites anuais UTC, merges, repetição de SHA, ausência de histórico, datas inconsistentes, recuperação após falhas, invalidação do cache e ausência de código ou evidências posteriores ao corte. O piloto percorre até 40 candidatos, com revisão das transições e primeiras evidências dos projetos antigos e registro de tempo, armazenamento e cobertura. Somente após a validação percorre-se a lista congelada de até 500 candidatos por coorte. O encerramento depende da lista, nunca de valores CK, da prevalência de IA ou da direção dos resultados.

Na inspeção do piloto, a presença de módulos de exemplo com dependências Boot em Spring Security evidenciou uma limitação na identificação do próprio framework. A execução revisada reconhece a combinação entre grupo Maven/Gradle declarado pelo projeto e fontes de produção no namespace Spring. A amostra e as referências Git são mantidas, o piloto é repetido com novo fingerprint e os valores CK nos mesmos SHA são conferidos. A tentativa anterior e a justificativa da emenda permanecem preservadas.

4.8. Ameaças à validade

Seleção e cobertura. A descoberta depende de tópicos e linguagem observados nos metadados atuais. Projetos desaparecidos ou privados não são recuperados, e configurações antigas ou dinâmicas podem não ser reconhecidas. A retenção de inativos e arquivados reduz uma seleção explícita por atividade, mas não elimina essas perdas. O limiar de tamanho, as falhas de extração e a confirmação técnica por snapshot alteram a cobertura temporal. A amostra elegível não representa automaticamente todos os projetos Spring.

Medição. Migrações e reescrita de histórico limitam a reconstrução temporal. Dependências não resolvidas podem afetar medidas estáticas, e a ferramenta CK documenta limites de resolução de tipos e chamadas [Aniche 2015]. As métricas Java não medem diretamente configuração XML/YAML, comportamento em execução, defeitos ou esforço de manutenção. Assim, mudanças relevantes à adoção de Boot podem ocorrer fora das dimensões medidas. A detecção de IA, por sua vez, pode omitir usos sem vestígios ou encontrar configurações sem uso efetivo.

Interpretação. Tamanho, idade, domínio, composição da equipe e mudanças de framework são explicações concorrentes para as variações. A composição dos conjuntos anuais e a repetição de versões também influenciam as sínteses. Os pares antes/depois mantêm a identidade do projeto, mas não controlam todas as mudanças ocorridas no intervalo, sobretudo quando há lacunas de vários anos. Não se estimam efeitos causais nem uma nota de maturidade, e a ausência de uma tendência uniforme não demonstra ausência de evolução em projetos específicos.

4.9. Relação com os pilotos anteriores

Os pilotos transversais de setembro e outubro de 2026 apoiaram a validação da detecção e do extrator. Eles mediram código atual e agruparam projetos pela criação. Seus testes entre grupos independentes, pareamentos e análises exploratórias não são utilizados como validação do acompanhamento longitudinal. O detalhamento anterior permanece em `Artigo/Arquivo`, e os dados, gráficos e instrumentos foram preservados nas execuções de `metricas_ck`. A execução histórica tem amostra, corte e manifestos próprios.

4.10. Reprodução e revisão dos autores

Os artefatos da execução preservam a população, a lista sorteada, as referências Git, os planos de snapshots, os dados brutos CK, as evidências de framework e IA e os diagnósticos. O repositório versiona instrumentos, amostras, tabelas consolidadas, relatórios e manifestos. Clones e arquivos volumosos da coleta permanecem no ambiente local; sua restauração é necessária para repetir a auditoria dos dados brutos e a extração exata. Os comandos e os limites do pacote versionado estão nos arquivos README do instrumento e dos artefatos históricos.

A revisão dos objetos Git foi assistida por IA, com referências concretas a SHA e arquivos; não constitui uma classificação independente por dois pesquisadores. A validação automatizada verifica regras e consistência dos cálculos, mas não substitui a avaliação humana da interpretação dos sinais. Permanece necessária uma revisão independente pelos autores dos casos de transição e de uma amostra dos negativos, com registro das divergências e de sua resolução. Essa etapa é uma recomendação para a submissão, não uma atividade já concluída.

5. Resultados e Discussão

Os resultados apresentam a cobertura da coleta e, em seguida, as trajetórias e os pares disponíveis. As contagens de candidatos, projetos elegíveis e observações anuais têm funções diferentes: o alcance da coleta não é o tamanho da amostra de cada comparação. As médias históricas utilizam a data do snapshot, enquanto a criação do repositório define sua coorte.

5.1. Da descoberta à amostra congelada

A reaplicação dos filtros à população original de 95.894 registros, preservando inativos e arquivados, resultou em 70.280 candidatos da base Boot: 30 em E1, 3.831 em E2, 20.380 em E3 e 46.039 em E4. A união com as consultas adicionais reuniu 117.621 identificadores únicos. Os filtros de metadados mantiveram 84.633 candidatos, dos quais um foi criado antes das coortes do estudo.

Os 84.632 candidatos nas coortes distribuem-se em 116 de E1, 5.836 de E2, 25.819 de E3 e 52.861 de E4. Entre os 116 antigos, 14 estão arquivados e 54 têm atividade antiga; esses atributos não os excluíram. O sorteio congelou 1.616 candidatos: todos os 116 de E1 e 500 de cada outra coorte. A confirmação de Spring, a disponibilidade histórica e o corte de tamanho foram verificados por snapshot após essa seleção.

5.2. Validação anterior à ampliação

O piloto percorreu os 40 candidatos previstos, sem substituição. Seus 720 cortes anuais incluíram 347 snapshots com árvore disponível, dos quais 119, pertencentes a 12 projetos, satisfizeram os critérios principais. A cobertura detalhada está no Apêndice B. A inspeção identificou o próprio Spring Security entre os candidatos: exemplos com dependência Boot não justificavam tratá-lo como uma aplicação usuária do framework. A correção manteve a amostra, os SHA e as métricas dos 720 registros; a tentativa anterior permaneceu preservada.

A auditoria automatizada recalculou as médias dos arquivos brutos e verificou os cortes temporais, sem divergências. Os 72 testes daquela versão passaram. A inspeção de evidências foi assistida por IA e não equivale a uma classificação independente pelos autores. Após essa validação, a coleta percorreu os demais candidatos da lista congelada.

Os pilotos transversais anteriores mediam código atual e agrupavam projetos pela criação. Eles contribuíram para o desenvolvimento dos instrumentos, mas seus contrastes não demonstram evolução histórica e não são combinados com os resultados a seguir. Seu detalhamento permanece em `Artigo/Arquivo` e nos artefatos de `metricas_ck`.

5.3. Execução histórica ampliada

A coleta percorreu os 1.616 candidatos congelados, sem reposição. Foram registrados 29.088 cortes anuais, incluindo anos anteriores à existência do histórico: 17.087 sem snapshot, 11.850 com extração CK concluída, 62 sem fontes Java, 30 com erro de extração, cinco com inconsistências temporais e 54 com falha de recuperação. Os últimos correspondem aos 18 cortes de três repositórios indisponíveis, também verificados pelo identificador na API do GitHub. Um retorno 404 não permite distinguir remoção de mudança de visibilidade.

A confirmação do framework e os critérios por snapshot retiveram **388 projetos e 2.533 observações anuais**. São versões repetidas dos mesmos projetos, não 2.533 observações independentes. A Tabela 2 informa a retenção por coorte. Os 116 candidatos antigos foram todos processados; 52 possuem ao menos uma versão elegível, somando 609 observações. As outras coortes oferecem contexto e não alteram esse conjunto principal.

A extração CK concluída não basta para tornar uma versão elegível: a comparação também exige confirmação do framework, porte mínimo e cobertura adequada. Por sua vez, um corte sem snapshot pode anteceder a existência do projeto, sem representar uma falha da ferramenta. Essas distinções impedem interpretar todos os registros fora da comparação principal como erros de coleta ou projetos de baixa qualidade.

Tabela 2. Cobertura da execução histórica, por coorte de criação.

Coorte	Candidatos	Projetos elegíveis	Snapshots elegíveis
E1	116	52	609
E2	500	105	1.001
E3	500	86	604
E4	500	145	319
Total	1.616	388	2.533

A auditoria automatizada validou a lista sorteada, os 18 registros por candidato, os limites temporais e as médias recalculadas dos dados brutos, incluindo produção e testes. Foram 3.142 extrações distintas, reutilizadas quando as fontes Java eram idênticas. A ampliação após o piloto consumiu aproximadamente 109 minutos com quatro trabalhadores, nas condições de cache da execução; clones e extrações ocuparam, respectivamente, 13,18 GiB e cerca de 598 MiB.

Emenda de classificação de framework. Um complemento de leitura de catálogos Gradle corrigiu 17 classificações anuais: dez no CAS e sete em RobotsJavaBackend. O complemento foi aplicado a toda a amostra, sem seleção pelo resultado CK. O segundo projeto permaneceu abaixo do tamanho mínimo; oito versões adicionais do CAS tornaram-se elegíveis e duas continuaram excluídas por submódulos não inspecionados. A classificação passou a reconhecer Boot no CAS desde 2016, após Spring tradicional em 2015, em vez de apenas em 2026. As métricas, os SHA, a amostra e as evidências de IA permaneceram idênticos nos 29.088 registros. Os manifestos conservam a emenda e a referência ao produtor CK original. Após a revisão da síntese para médias, os 88 testes passaram.

5.4. Trajetórias dos projetos criados entre 2009 e 2013

A Tabela 3 descreve exclusivamente a coorte antiga. Cada projeto contribui uma vez por época, pela média de seus snapshots elegíveis. E1 reúne 33 projetos e E4 reúne 43; esses conjuntos não são idênticos. Portanto, diferenças entre linhas combinam possíveis mudanças de código e de composição, não sendo uma comparação pareada dos mesmos projetos entre épocas.

Tabela 3. Coorte antiga: médias por época do código, com produção e testes.

Época	Projetos	Snapshots	CBO	RFC	WMC	LOC/método	DIT	NOC	LCOM*
E1	33	71	5,758	8,111	8,781	5,985	1,346	0,201	0,289
E2	43	155	5,934	8,552	9,155	6,127	1,329	0,161	0,295
E3	47	168	6,068	8,525	8,889	6,252	1,344	0,177	0,295
E4	43	172	6,238	8,838	9,029	6,173	1,317	0,173	0,289
2026 parcial	43	43	6,358	9,103	9,103	6,180	1,311	0,166	0,288

Entre E1 e E4, as médias passam pelos seguintes valores: CBO, de 5,758 para 6,238; RFC, de 8,111 para 8,838; WMC, de 8,781 para 9,029; LOC por método, de 5,985 para 6,173; DIT, de 1,346 para 1,317; NOC, de 0,201 para 0,173; LCOM*, de 0,2894 para 0,2890. Os indicadores variam em direções distintas. Esses valores não demonstram piora geral por haver aumentos, nem melhoria geral por haver reduções: tamanho, responsabilidades e composição dos projetos participam de sua interpretação.

A estabilidade também precisa ser interpretada: em 2025, 20 dos 43 snapshots elegíveis da coorte antiga repetem o SHA do corte anterior; em 2026, são 23 de 43. Assim, a distribuição inclui código legado mantido disponível, além de projetos em evolução ativa. A Figura 1 separa a média anual da amostra de contexto das trajetórias individuais antigas. Lacunas são conservadas nas tabelas; segmentos entre pontos não significam observações intermediárias.

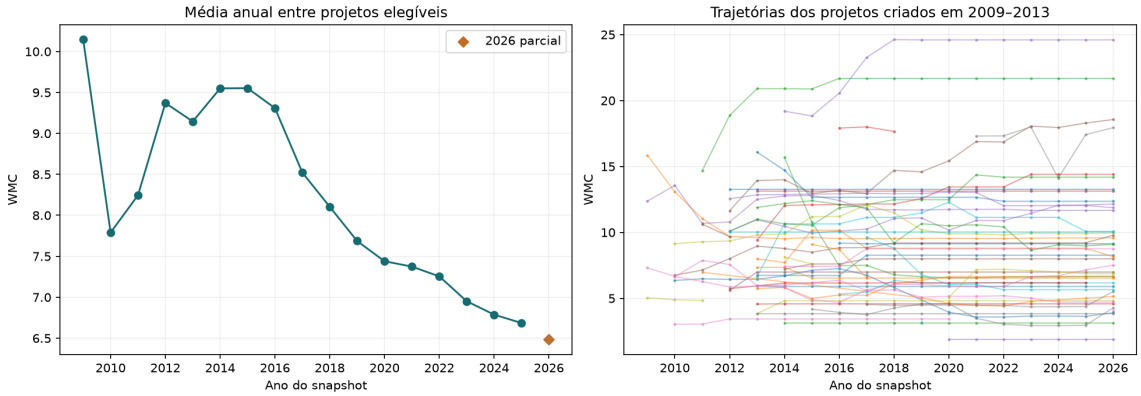


Figura 1. WMC no histórico: média anual dos elegíveis de todas as coortes (esquerda) e trajetórias individuais da coorte antiga (direita). O período de 2026 é parcial. Mudanças da composição afetam as médias anuais.

5.5. Mudanças que acompanharam Spring Boot

Na coorte antiga, 16 projetos possuem versões elegíveis antes e depois da primeira presença anual observável de Boot, em anos completos. A Tabela 4 mostra médias antes/depois e, separadamente, a média das diferenças calculadas projeto a projeto. Para os mesmos pares e pesos, a média das diferenças corresponde à média depois menos a média antes.

Tabela 4. Spring Boot na coorte antiga: mesmos 16 projetos antes e depois ($n_a = n_b = 16$).

Métrica	Média antes	Média depois	Média das diferenças
CBO	5,883	6,227	0,3447
RFC	9,277	9,490	0,2132
WMC	10,091	9,670	-0,4208
LOC/método	6,257	6,203	-0,0541
DIT	1,353	1,346	-0,0069
NOC	0,103	0,102	-0,0003
LCOM*	0,306	0,310	0,0043

A média das mudanças é positiva para CBO, RFC e LCOM*, e negativa para WMC, LOC por método, DIT e NOC. As magnitudes descrevem esta amostra; não são testes de significância ou efeitos causais. Quatorze pares usam anos consecutivos, um tem intervalo de dois anos e o Causeway tem intervalo de oito anos entre versões elegíveis. Esse último agrega muitas mudanças concorrentes. No conjunto das quatro coortes, existem 21 pares completos de Boot. As tabelas de contexto preservam separadamente os projetos sem transição observada nas mesmas janelas.

A redução média de WMC descreve menor complexidade agregada por classe nos pares, mas não informa se a maioria dos projetos apresentou redução. De modo semelhante, o aumento médio de CBO não identifica quais dependências mudaram nem se eram necessárias às novas responsabilidades. Como não foi realizada uma análise específica da influência do par com oito anos de intervalo, não se afirma que a direção das médias permaneceria sem ele. Os grupos sem transição fornecem contexto descritivo, não um controle experimental equivalente.

Os casos já inspecionados ajudam a interpretar o marco observado: Hawtio passa de Spring tradicional em 2013 para presença de Boot em 2014, e BroadleafCommerce apresenta essa sequência entre 2016 e 2017. Esses intervalos localizam a mudança entre versões anuais, sem isolar uma migração de todas as demais alterações. Já o caso CAS mostra que a data reconhecida depende da capacidade do detector de resolver as declarações do build; uma configuração não reconhecida não comprova ausência anterior de Boot.

Também há Boot preliminar antes de 2014: o snapshot de 2013 de `spring-guides/gs-rest-service` declara 0.5.0.M6. Com 43 LOC, ele fica fora da comparação principal, mas permanece registrado. Isso não contradiz o lançamento estável de abril de 2014.

5.6. Análise exploratória das evidências de IA

Onze projetos antigos apresentam evidência em alguma versão elegível. Apenas um possui par em anos completos até 2025: OpenESPI-DataCustodian, de 2024 para 2025. A evidência de coautoria Claude foi integrada em 1ž de julho de 2025; a data de autoria é 24 de junho. No mesmo ano aparece Boot, impedindo separar essas mudanças. CBO passa de 9,895 para 12,180 e WMC de 11,140 para 10,079. Esse caso isolado não representa o comportamento geral de projetos assistidos por IA.

Os outros dez pares antigos usam 2025 e o corte parcial de 2026, fixado em 11 de outubro de 2026 às 19:34:13 UTC. A Tabela 5 mantém esse acompanhamento separado. Sinais como identidades de bot, arquivos de instruções e entradas de gitignore não determinam quanto código foi gerado nem demonstram uso contínuo.

No piloto, MyBatis/Spring apresentou uma identidade de bot Copilot, enquanto Hawtio apresentou um arquivo `AGENTS.md`. O primeiro sinal se refere à identidade registrada no histórico; o segundo, a instruções presentes no projeto. A reunião desses casos permite estudar a presença de vestígios, mas não estabelece uma exposição uniforme à assistência. Além disso, mudanças medidas entre o encerramento de 2025 e o corte parcial de 2026 podem anteceder a primeira evidência ou ocorrer depois dela dentro do mesmo intervalo.

Tabela 5. IA na coorte antiga: dez pares com acompanhamento parcial em 2026 ($n_a = n_b = 10$).

Métrica	Média antes	Média parcial	Média das diferenças
CBO	5,829	5,878	0,0489
RFC	9,460	9,958	0,4983
WMC	7,371	7,763	0,3913
LOC/método	6,924	6,994	0,0703
DIT	1,417	1,419	0,0020
NOC	0,159	0,139	-0,0205
LCOM*	0,232	0,232	0,0006

Na amostra de todas as coortes, há quatro pares de IA em anos completos e 17 pares adicionais no período parcial. As contagens são menores que o total de positivos porque nem todo projeto tem uma versão anterior elegível sem evidência. Esses resultados permitem descrever os casos, mas não sustentar uma conclusão geral de melhora ou

piora associada à IA. A ausência dos sinais procurados também não estabelece um grupo comprovadamente sem uso.

5.7. Interpretação conjunta

O acompanhamento histórico mostra que permanência, mudança estrutural e adoção de ferramentas são fenômenos distintos. Um projeto pode repetir código por vários anos, incorporar Boot em um intervalo e apresentar vestígios de IA muito depois. O valor do desenho está em localizar essas observações em versões verificáveis, evitando atribuir uma tecnologia atual a toda a história do repositório.

As direções distintas das métricas não constituem um teste que refute o progresso da comunidade. Elas delimitam o que esta descrição permite afirmar sobre a amostra, diante de mudanças de composição e de contexto. A resolução anual e os indicadores Java também não capturam todas as alterações de configuração, funcionalidade ou esforço. As conclusões a seguir respondem às perguntas do estudo nesse alcance.

6. Conclusão

Este trabalho caracterizou versões históricas de projetos públicos do ecossistema Spring, com foco nos criados entre 2009 e 2013. A execução percorreu 1.616 candidatos e reuniu 2.533 observações anuais elegíveis de 388 projetos. A coorte principal contribuiu com 52 projetos e 609 observações. A contribuição é um acompanhamento rastreável de dimensões estruturais e marcos observáveis, com versões, cobertura e limitações associados às medições.

Q1: evolução estrutural. As trajetórias e sínteses registram mudanças em acoplamento, complexidade, tamanho, herança e coesão, além da permanência de código sem novos commits. As médias por época variam em direções distintas e reúnem conjuntos parcialmente diferentes de projetos. O estudo não estabelece uma melhoria uniforme da amostra nem permite extrapolar uma medida de maturidade para a comunidade. Essa limitação não elimina a possibilidade de evolução favorável em projetos ou dimensões específicos.

Q2: presença observável de Spring Boot. Os 16 pares da coorte antiga em anos completos apresentam redução média de WMC e LOC por método e aumento médio de CBO, RFC e LCOM*, acompanhados de reduções de DIT e NOC. Esses movimentos descrevem os pares disponíveis; não equivalem, isoladamente, a melhora ou piora arquitetural. Os intervalos variáveis e as mudanças concorrentes impedem atribuir as diferenças ao Boot. O estudo também não mede diretamente possíveis alterações em configuração XML/YAML ou esforço de desenvolvimento.

Q3: evidências de assistência por IA. A coorte antiga fornece apenas um par em anos completos e dez pares com acompanhamento parcial em 2026. No único par completo, a presença observável de Boot coincide com o ano da evidência de IA. Portanto, os dados permitem descrever casos e diferenças temporais, mas não sustentar uma conclusão geral sobre benefício ou prejuízo da IA. Vestígios de identidade, coautoria e configuração têm alcances diferentes; a ausência dos sinais procurados não confirma ausência de uso.

Q4: contexto entre projetos. As demais coortes e os estratos de tamanho e framework situam as trajetórias no conjunto observado. Essas comparações mostram o contexto da amostra, sem formar grupos experimentalmente equivalentes ou separar todos os efeitos de idade, domínio e composição. Comparações entre projetos e mudanças dentro do mesmo repositório devem, assim, permanecer identificadas separadamente.

Como desdobramentos, recomenda-se comparar épocas adjacentes usando os projetos presentes em ambos os períodos, descrever a distribuição das mudanças individuais e examinar a influência de valores extremos e de intervalos longos. Também são necessárias análises de sensibilidade ao limiar de tamanho e uma revisão humana independente das transições e de uma amostra dos negativos. Essas atividades permanecem fora da execução apresentada. A ampliação do acompanhamento de IA deve preservar o protocolo e a distinção entre anos completos e períodos parciais, sem selecionar projetos em função da direção das métricas.

Referências

Aniche, M. (2015). Java Code Metrics Calculator (CK). `https://github.com/mauricioaniche/ck`. Versao 0.7.1.

Aniche, M., Treude, C., Zaidman, A., van Deursen, A., and Gerosa, M. A. (2016). SATT: Tailoring code metric thresholds for different software architectures. In *2016 IEEE 16th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pages 41–50. IEEE.

Basili, V. R., Caldiera, G., and Rombach, H. D. (1994). The goal question metric approach. In *Encyclopedia of Software Engineering*, pages 528–532. John Wiley & Sons, New York.

Chidamber, S. R. and Kemerer, C. F. (1994). A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493.

Child, M., Rosner, P., and Counsell, S. (2019). A comparison and evaluation of variants in the coupling between objects metric. *Journal of Systems and Software*, 151:120–132.

Dey, T., Mousavi, S., Ponce, E., Godfrey, T., Padhy, B., Nguyen, A., and Mockus, A. (2020). Detecting and characterizing bots that commit code. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR 2020)*, pages 209–219, Seoul, Republic of Korea. ACM.

Git (2026). git-rev-list documentation. `https://git-scm.com/docs/git-rev-list`. Opção –first-parent. Consultado em 11 de outubro de 2026.

GitHub (2021). Introducing github copilot: your ai pair programmer. `https://github.blog/news-insights/product-news/introducing-github-copilot-ai-pair-programmer/`. Publicado em 29 de junho de 2021.

Godfrey, M. W. and Tu, Q. (2000). Evolution in open source software: A case study. In *Proceedings of the International Conference on Software Maintenance*. IEEE.

Hassan, A. E. (2008). The road ahead for Mining Software Repositories. In *2008 Frontiers of Software Maintenance*, pages 48–57. IEEE.

ISO/IEC (2011). Systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models. International Standard ISO/IEC 25010:2011, International Organization for Standardization, Geneva.

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. (2014). The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR '14)*, pages 92–101, Hyderabad, India. ACM.

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. (2016). An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering*, 21(5):2035–2071.

Liu, J., Xia, C. S., Wang, Y., and Zhang, L. (2023). Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. Curran Associates.

Moradi Dakhel, A., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M. C., and Jiang, Z. M. (2023). GitHub Copilot AI pair programmer: asset or liability? *Journal of Systems and Software*, 203:111734.

Peng, S., Kalliamvakou, E., Cihon, P., and Demirer, M. (2023). The impact of AI on developer productivity: evidence from GitHub Copilot. arXiv:2302.06590.

Pressman, R. S. and Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, New York, 9 edition.

Radjenović, D., Heričko, M., Torkar, R., and Živković, A. (2013). Software fault prediction metrics: a systematic literature review. *Information and Software Technology*, 55(8):1397–1418.

Robbes, R., Matricon, T., Degueule, T., Hora, A., and Zacchiroli, S. (2026). Agentic much? adoption of coding agents on GitHub. arXiv:2601.18341v2. Versão de 8 de abril de 2026. Consultado em 11 de outubro de 2026.

Singh, Y., Kaur, A., and Malhotra, R. (2010). Empirical validation of object-oriented metrics for predicting fault proneness models. *Software Quality Journal*, 18(1):3–35.

Spring (2014). Spring boot 1.0 ga released. `https://spring.io/blog/2014/04/01/spring-boot-1-0-ga-released/`. Publicado em 1 de abril de 2014.

Sun, P., Kim, D.-K., Hua, M., and Lu, L. (2022). Measuring impact of dependency injection on software maintainability. *Computers*, 11(9):141.

Yetistiren, B., Ozsoy, I., Ayerdem, M., and Tuzun, E. (2023). Evaluating the code quality of AI-assisted code generation tools: an empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. arXiv:2304.10778.

Apêndice A — Visualizações históricas e leitura dos resultados

Os gráficos de trajetória usam o ano do snapshot no eixo horizontal e uma medida CK por painel. São apresentadas separadamente a média anual entre projetos e as linhas individuais dos projetos criados entre 2009 e 2013. O ponto parcial de 2026 é identificado;

lacunas de cobertura não são substituídas por zero. Mudanças na média anual podem decorrer da composição de projetos disponíveis, enquanto a trajetória individual acompanha o mesmo repositório.

As tabelas por época apresentam uma contribuição por repositório, com quantidades de projetos e snapshots. O total de projetos únicos não deve ser obtido pela soma das épocas, pois um mesmo projeto pode atravessá-las. Os contrastes por evidência distinguem presença observada, ausência de evidência e transição dentro da época.

Nas tabelas antes/depois, `n_a` é o número de projetos com medição anterior e `n_b` o número com medição posterior. Os pares correspondem aos mesmos repositórios. A diferença é calculada dentro de cada par antes de obter a média; para os mesmos pares e pesos, ela corresponde à média posterior menos a média anterior. Os SHA e as datas permitem conferir cada versão utilizada.

As sete medidas conservam suas unidades originais. Não se cria uma nota composta de maturidade. Não se apresentam p-valores, deltas de Cliff ou afirmações automáticas de melhoria. Arquivos CSV, gráficos PNG/SVG, referências fixadas, evidências, diagnósticos e hashes do instrumento integram o pacote de reprodução. Os relatórios dos pilotos transversais anteriores permanecem identificados separadamente.

Apêndice B — Cobertura do piloto histórico

O piloto compreendeu os primeiros dez candidatos sorteados de cada coorte. Dos 720 cortes anuais, 373 não tinham histórico disponível até o corte, 346 tiveram extração CK concluída e um não continha fontes Java. Os 347 snapshots com árvore disponível corresponderam a 127 extrações distintas, em razão da reutilização de fontes idênticas. A tabela conserva os resultados do piloto corrigido, anterior à ampliação.

Tabela 6. Cobertura do piloto histórico por coorte de criação.

Coorte	Candidatos	Snapshots disponíveis	Elegíveis	Projetos elegíveis
E1	10	155	77	5
E2	10	101	31	3
E3	10	75	8	1
E4	10	16	3	3
Total	40	347	119	12

A reexecução utilizou quatro trabalhadores e consumiu aproximadamente 355 segundos, reutilizando cerca de 505 MiB de clones e produzindo aproximadamente 44 MiB de extrações. Esses valores descrevem o ambiente e o cache daquela execução, sem estimar o custo de uma coleta inteiramente nova.